

Introducción a las ventanas de contexto

Módulo 3 — Context Engineering Profesional

Objetivos del capítulo

En este capítulo aprenderemos a:

- comprender cómo miden los modelos el contexto;
 - interpretar el concepto de token;
 - optimizar el uso de la ventana disponible;
 - aplicar técnicas de compresión y resumido;
 - diseñar aplicaciones escalables para conversaciones y documentos extensos.
-

¿Qué es una ventana de contexto?

Todo modelo de lenguaje posee un límite en la cantidad de información que puede procesar simultáneamente. Ese límite recibe el nombre de **ventana de contexto**.

Dentro de esa ventana conviven todos los elementos que el modelo necesita para generar una respuesta:

- instrucciones del sistema;
- historial conversacional;
- memoria recuperada;
- documentos obtenidos mediante RAG;
- resultados de herramientas;
- mensaje actual del usuario.

La capacidad de diseñar y administrar este espacio es una de las competencias centrales del Context Engineering.

Una analogía útil

Imagine un escritorio de trabajo.

Solo puede colocar una cantidad limitada de documentos sobre él. Si incorpora demasiadas hojas, deberá retirar algunas, resumir otras o reorganizar el espacio disponible.

La ventana de contexto funciona de manera similar: el modelo únicamente puede "ver" aquello que entra dentro de su capacidad máxima.

¿Por qué es importante?

Una gestión inadecuada de la ventana puede provocar:

- pérdida de información relevante;
- respuestas inconsistentes;
- aumento del costo por consulta;
- mayor latencia;
- degradación del rendimiento.

En aplicaciones empresariales, estos problemas suelen aparecer mucho antes de alcanzar el límite máximo de tokens.

¿Qué ocupa espacio?

No solo el mensaje del usuario consume tokens.

También ocupan espacio:

- las instrucciones del sistema;
- los ejemplos (few-shot);
- el historial;
- la memoria;
- los documentos recuperados;
- las respuestas previas del modelo.

Por este motivo, ampliar el contexto sin una estrategia suele empeorar los resultados.

Resumen

La ventana de contexto representa uno de los recursos más valiosos de cualquier LLM. Administrarla correctamente es tan importante como elegir el modelo adecuado.

En la próxima sección estudiaremos qué es un token y cómo influye en el funcionamiento interno de los modelos de lenguaje.

Tokens: la unidad fundamental del contexto

Módulo 3 — Context Engineering Profesional

Introducción

Cuando hablamos de una ventana de contexto de 128.000, 200.000 o un millón de tokens, no nos referimos a palabras ni a caracteres. Los modelos de lenguaje trabajan con una unidad propia denominada **token**.

Comprender qué es un token y cómo se contabiliza es imprescindible para diseñar aplicaciones eficientes.

¿Qué es un token?

Un token es la unidad mínima de información que procesa un modelo de lenguaje.

Dependiendo del idioma y del texto, un token puede representar:

- una palabra completa;
- parte de una palabra;
- un signo de puntuación;
- un número;
- un espacio;
- incluso un carácter individual.

Por esta razón, no existe una conversión exacta entre palabras y tokens.

Un ejemplo sencillo

Considere la siguiente frase:

"La inteligencia artificial transforma organizaciones."

Aunque contiene solo cinco palabras, el modelo podría dividirla en varios tokens adicionales según el algoritmo de tokenización utilizado.

Dos modelos distintos incluso pueden producir cantidades diferentes de tokens para el mismo texto.

¿Qué consume tokens?

Dentro de una solicitud al modelo consumen tokens:

- instrucciones del sistema;
- ejemplos (few-shot);
- historial conversacional;
- memoria recuperada;
- documentos RAG;
- resultados de herramientas;
- mensaje del usuario;
- respuesta generada por el modelo.

Es un error habitual pensar que únicamente cuenta el mensaje del usuario.

Tokens de entrada y de salida

Conviene distinguir dos conceptos:

Tokens de entrada

Son todos los tokens enviados al modelo como contexto. Incluyen el system prompt, el historial, los documentos recuperados y el mensaje actual.

Tokens de salida

Son los tokens utilizados para construir la respuesta generada por el modelo.

Muchos proveedores facturan ambos valores por separado, y los tokens de salida suelen tener un precio mayor que los de entrada. Una aplicación que genera respuestas muy extensas puede acumular costos significativos aunque el contexto de entrada sea compacto.

¿Por qué importan?

El número de tokens influye directamente en:

- el costo de cada consulta;
- la velocidad de respuesta;
- el consumo de recursos;
- la cantidad de información disponible para el razonamiento.

Una aplicación bien diseñada optimiza el uso de los tokens en lugar de intentar ocupar toda la ventana disponible.

Buenas prácticas

- Eliminar información redundante antes de enviar la solicitud.
 - Resumir conversaciones extensas en lugar de conservarlas íntegras.
 - Recuperar únicamente los documentos relevantes para la consulta actual.
 - Evitar ejemplos few-shot innecesarios cuando el modelo no los requiere.
 - Medir periódicamente el consumo de tokens durante el desarrollo y la operación.
-

Resumen

Los tokens constituyen la unidad básica con la que trabajan los modelos de lenguaje. Todo lo que incorporamos al contexto se transforma en tokens y compete por un espacio limitado dentro de la ventana de contexto.

En la próxima sección estudiaremos cómo los modelos tokenizan el texto y por qué diferentes proveedores pueden producir cantidades distintas de tokens para el mismo contenido.

Cómo funciona la tokenización

Módulo 3 — Context Engineering Profesional

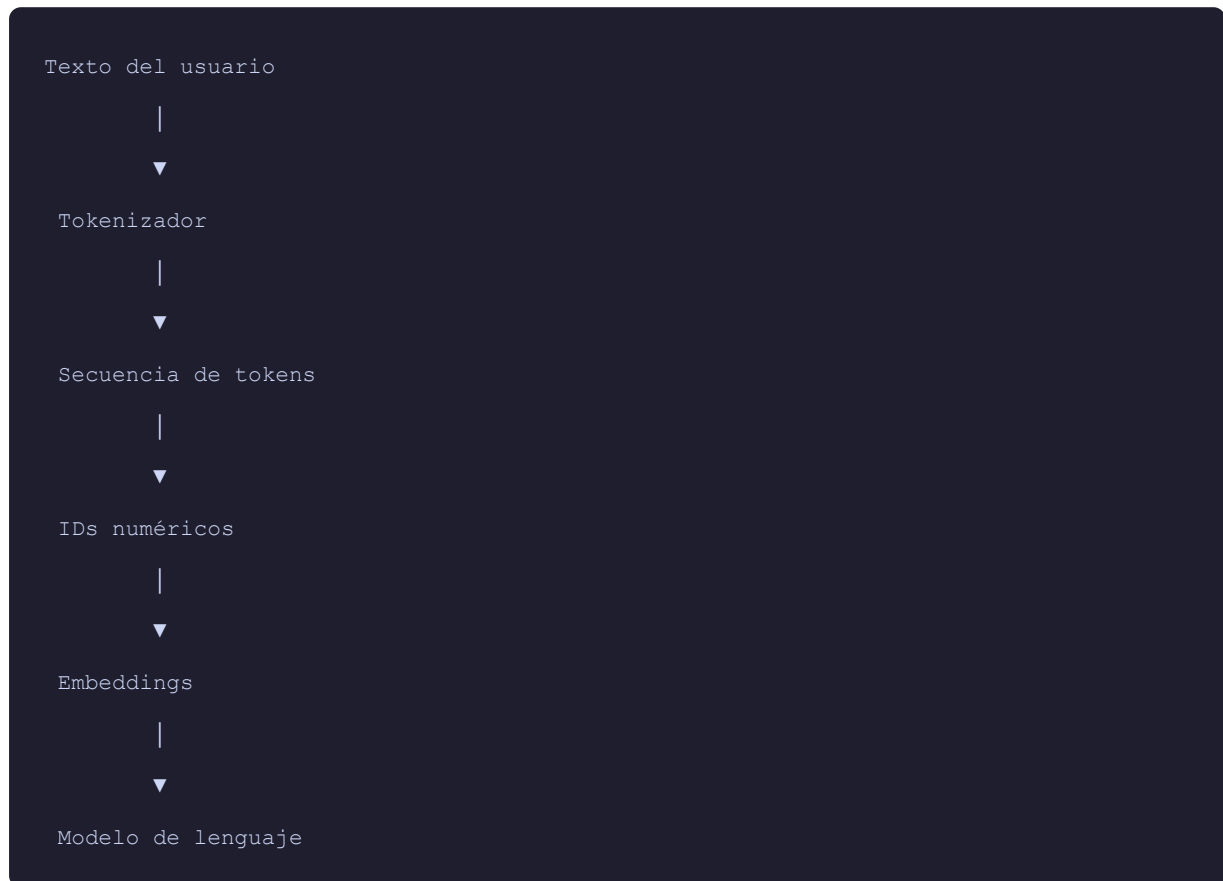
Introducción

Antes de que un modelo pueda interpretar una solicitud, debe convertir el texto en una representación que pueda procesar matemáticamente. Ese proceso se conoce como **tokenización**.

La tokenización transforma una secuencia de caracteres en una secuencia de tokens, que posteriormente serán convertidos en identificadores numéricos (IDs) y, finalmente, en vectores utilizados durante la inferencia.

Del texto al modelo

De forma simplificada, el recorrido de una consulta es el siguiente:



El modelo nunca "lee" palabras directamente. Todo el procesamiento ocurre sobre representaciones numéricas.

¿Por qué no se utilizan palabras?

Podría parecer más sencillo dividir el texto por espacios, pero ese enfoque presenta numerosos problemas:

- palabras compuestas;
- distintos idiomas;
- errores ortográficos;
- abreviaturas;
- emojis;
- signos de puntuación;
- nombres propios.

Los tokenizadores modernos utilizan algoritmos como Byte Pair Encoding (BPE) o SentencePiece, que encuentran unidades reutilizables para representar eficientemente millones de textos diferentes.

Diferentes modelos, diferentes tokenizadores

Cada familia de modelos suele incorporar su propio tokenizador.

Como consecuencia:

- un mismo párrafo puede generar cantidades distintas de tokens;
- el costo de procesarlo puede variar;
- la ventana de contexto efectiva también puede cambiar.

Por este motivo, las estimaciones de consumo deben realizarse sobre el modelo concreto que utilizará la aplicación.

Ejemplo conceptual

Supongamos la palabra:

internacionalización

Un tokenizador podría dividirla en varias partes reutilizables —como "inter", "nacional", "ización"— mientras que otro podría representarla con una combinación completamente diferente.

El objetivo no es preservar las palabras originales, sino obtener una representación eficiente para el aprendizaje y la inferencia.

Implicancias para el AI Engineer

Comprender la tokenización permite:

- estimar con mayor precisión el consumo de contexto;
- diseñar prompts más eficientes;
- reducir costos de operación;
- evitar superar el límite de la ventana de contexto.

Aunque normalmente no será necesario implementar un tokenizador, sí resulta esencial conocer su impacto sobre la arquitectura.

Buenas prácticas

- Medir el consumo de tokens durante el desarrollo con las herramientas propias de cada proveedor.
 - No asumir equivalencias fijas entre palabras y tokens.
 - Probar el comportamiento con el modelo definitivo, no con aproximaciones genéricas.
 - Evitar incorporar información redundante al contexto.
-

Resumen

La tokenización constituye el primer paso del procesamiento de cualquier modelo de lenguaje. Convertir correctamente el texto en tokens permite representar información compleja de forma eficiente y determina, en gran medida, el costo y la capacidad de una aplicación basada en LLM.

En la próxima sección analizaremos cómo evolucionaron las ventanas de contexto y qué capacidades ofrecen los modelos modernos.

La evolución de las ventanas de contexto

Módulo 3 — Context Engineering Profesional

Introducción

Las primeras generaciones de modelos de lenguaje podían procesar únicamente pequeñas cantidades de texto. Esto obligaba a resumir conversaciones, dividir documentos y fragmentar problemas complejos.

En pocos años, las ventanas de contexto crecieron desde unos pocos miles de tokens hasta cientos de miles e incluso millones en algunos modelos. Este cambio modificó profundamente la forma de diseñar aplicaciones basadas en IA.

Una breve evolución

De forma general, la evolución ha seguido esta tendencia:

- **Primeros modelos:** ventanas reducidas, adecuadas para tareas simples y consultas cortas. Las aplicaciones debían fragmentar cualquier documento antes de enviarlo.
- **Segunda generación:** capacidad suficiente para documentos extensos y conversaciones prolongadas. Aparece la posibilidad de enviar contratos, artículos o sesiones de soporte completas en una sola solicitud.
- **Modelos actuales:** procesamiento de grandes bases documentales, múltiples archivos y sesiones de trabajo complejas. Algunos modelos superan el millón de tokens de contexto.

Más importante que el número absoluto de tokens es la posibilidad de mantener información relevante durante más tiempo sin necesidad de reconstruir el estado de la conversación en cada solicitud.

¿Más contexto siempre es mejor?

No necesariamente.

Una ventana más grande ofrece ventajas evidentes:

- menos necesidad de resumir;
- mayor continuidad conversacional;
- posibilidad de analizar documentos completos;
- mejor soporte para tareas complejas.

Sin embargo, también introduce desafíos:

- mayor costo por consulta;
- incremento de la latencia;
- más información irrelevante compitiendo por la atención del modelo;
- mayor complejidad para construir un contexto de calidad.

El objetivo del AI Engineer no es llenar la ventana, sino utilizarla de manera inteligente.

El fenómeno "lost in the middle"

Ampliar la ventana de contexto no garantiza que el modelo utilice toda la información con la misma eficacia.

Investigaciones recientes han documentado un fenómeno conocido como **"lost in the middle"**: los modelos tienden a prestar mayor atención a la información ubicada al principio y al final del contexto, mientras que los datos situados en el centro del documento reciben menos peso durante el razonamiento.

Las consecuencias prácticas son significativas:

- colocar instrucciones críticas a mitad del system prompt puede degradar los resultados;
- insertar los documentos más relevantes en el centro de un contexto muy largo puede hacer que el modelo los subutilice;
- una conversación extensa puede provocar que el modelo "olvide" compromisos asumidos en turnos intermedios aunque estos sigan siendo parte del contexto visible.

La respuesta arquitectónica al "lost in the middle" es diseñar el orden del contexto de forma deliberada: las instrucciones más importantes van al principio, los documentos más relevantes al inicio o al final, y la información de menor prioridad en el centro.

Calidad antes que cantidad

Agregar información indiscriminadamente rara vez mejora los resultados.

Por ejemplo, incorporar cien páginas de documentación cuando solo dos contienen la respuesta puede dificultar el razonamiento del modelo: introduce ruido, aumenta el costo y, combinado con el fenómeno "lost in the middle", puede hacer que el modelo produzca respuestas menos precisas que si hubiera recibido solo las dos páginas relevantes.

Una arquitectura madura prioriza:

- información pertinente;
 - contexto actualizado;
 - datos sin duplicación;
 - instrucciones claras ubicadas en posiciones privilegiadas.
-

Impacto en la arquitectura

La ampliación de las ventanas de contexto permitió construir soluciones que antes eran impracticables:

- asistentes empresariales con acceso a grandes manuales;
- análisis de contratos extensos;
- revisión de repositorios de código;
- agentes que coordinan múltiples herramientas;
- procesamiento de informes completos.

No obstante, estas aplicaciones siguen necesitando mecanismos como RAG, memoria y filtrado de contexto para mantener un buen rendimiento. Una ventana grande sin estrategia de selección convierte el problema de la capacidad en un problema de calidad.

Buenas prácticas

- Diseñar para la calidad del contexto, no para su tamaño.
 - Colocar las instrucciones y los documentos más importantes al principio del contexto.
 - Medir el costo asociado a ventanas extensas antes de adoptar modelos con mayor capacidad.
 - Recuperar únicamente la información necesaria para cada consulta.
 - Combinar ventanas amplias con estrategias de Context Engineering.
-

Resumen

La evolución de las ventanas de contexto amplió enormemente las capacidades de los modelos de lenguaje. Sin embargo, una ventana mayor no elimina la necesidad de diseñar cuidadosamente el contexto: fenómenos como el "lost in the middle" demuestran que el orden y la selección importan tanto como la capacidad total. La diferencia entre una aplicación correcta y una excelente continúa estando en la arquitectura que construye el AI Engineer.

En la próxima sección analizaremos qué ocurre cuando el contexto supera la capacidad del modelo y qué estrategias permiten administrar esa limitación.

Cuando el contexto supera la ventana disponible

Módulo 3 — Context Engineering Profesional

Introducción

Toda ventana de contexto, por grande que sea, tiene un límite. Tarde o temprano una conversación extensa, un conjunto de documentos o una combinación de herramientas y memoria terminarán superando esa capacidad.

El desafío del AI Engineer consiste en decidir **qué información conservar, cuál resumir y cuál descartar**, maximizando la calidad del contexto sin exceder el límite del modelo.

¿Qué ocurre al superar el límite?

Cuando el contexto excede la capacidad admitida por el modelo, pueden producirse distintas situaciones dependiendo del proveedor:

- rechazo de la solicitud con un error de límite excedido;
- truncamiento automático del contexto por el lado del proveedor;
- eliminación de los mensajes más antiguos del historial;
- necesidad de reconstruir el contexto antes de reenviar la consulta.

En cualquier caso, el modelo no podrá razonar sobre información que ya no forme parte de la ventana. Una arquitectura que no gestiona activamente este límite dejará que el proveedor tome esa decisión, generalmente con resultados menos precisos que una política diseñada deliberadamente.

El problema de las conversaciones largas

Imagine un asistente que acompaña a un usuario durante varias semanas.

Cada interacción agrega:

- nuevos mensajes del usuario y respuestas del modelo;
- resultados de herramientas ejecutadas;
- documentos recuperados para responder consultas específicas;
- instrucciones adicionales incorporadas en distintos momentos.

Si todo permanece dentro del contexto, llegará un momento en que el modelo no podrá procesarlo completamente.

Por este motivo, las aplicaciones profesionales nunca dependen exclusivamente del historial conversacional. Requieren una política activa de administración del contexto.

Estrategias de administración

Existen varias técnicas para mantener el contexto dentro de límites razonables.

1. Descarte selectivo

Eliminar mensajes o fragmentos que ya no aportan valor para las consultas futuras.

Candidatos habituales al descarte:

- saludos y cortesías de inicio de sesión;
- mensajes repetidos o reformulaciones del mismo pedido;
- respuestas del modelo ya superadas por versiones posteriores;
- resultados de herramientas de una tarea ya concluida.

2. Resumen

Reemplazar múltiples interacciones por un resumen estructurado que conserve únicamente los hechos relevantes: decisiones adoptadas, compromisos asumidos, datos confirmados y estado actual del problema.

El resumen puede generarse con el mismo modelo o con uno más económico dedicado a esa tarea.

3. Recuperación bajo demanda

En lugar de mantener toda la información en el contexto activo, almacenarla fuera y recuperarla cuando realmente sea necesaria mediante mecanismos como RAG.

Esta estrategia es especialmente efectiva para bases de conocimiento estables: manuales, políticas, documentación técnica.

4. Memoria persistente

Guardar únicamente conocimientos duraderos y reutilizables —preferencias del usuario, acuerdos de largo plazo, perfil del cliente— evitando conservar eventos transitorios que solo son relevantes durante una sesión.

Elegir qué conservar

Una pregunta útil durante el diseño es:

"¿El modelo necesitará esta información para responder correctamente dentro de diez minutos?"

Si la respuesta es negativa, probablemente no deba permanecer en el contexto activo. Si la respuesta es "tal vez", la estrategia correcta es almacenarla fuera del contexto y recuperarla en caso necesario.

Caso práctico

Supongamos un asistente de soporte que atiende un incidente durante varios días.

Sin gestión activa del contexto, el historial crece linealmente hasta superar la ventana del modelo.

Una arquitectura eficiente podría:

1. mantener únicamente las últimas cinco a diez interacciones en el contexto activo;
2. resumir las decisiones y acciones tomadas en un bloque de estado que se actualiza al cierre de cada sesión;
3. recuperar documentación técnica relevante mediante RAG en el momento de cada consulta;
4. conservar en memoria persistente únicamente las preferencias del usuario y el historial de incidentes resueltos.

El resultado es un contexto compacto, relevante y mucho más eficiente que acumular días de conversación sin filtrado.

Buenas prácticas

- Diseñar una política explícita de descarte desde el inicio del proyecto.
 - Resumir el contexto antes de alcanzar el límite del modelo, no después.
 - Evitar duplicar información entre el historial, la memoria y los documentos RAG.
 - Medir el crecimiento del contexto durante las pruebas de carga.
 - Tratar la ventana como un recurso limitado, no como un almacén ilimitado.
-

Resumen

La administración del contexto es una tarea continua, no una operación puntual. Las aplicaciones modernas combinan descarte, resumido, memoria y recuperación inteligente para mantener conversaciones extensas sin comprometer la calidad de las respuestas.

En la próxima sección estudiaremos las técnicas de resumido y compresión que permiten reducir el tamaño del contexto preservando la información esencial.

Técnicas de resumido y compresión del contexto

Módulo 3 — Context Engineering Profesional

Introducción

Las aplicaciones modernas no esperan a que la ventana se llene para actuar. Implementan estrategias de compresión progresiva que mantienen el contexto dentro de límites operativos sin sacrificar la continuidad de la conversación.

Resumir no es perder información: es preservar lo esencial eliminando lo prescindible. La calidad de un resumen depende tanto del algoritmo utilizado como del criterio empleado para decidir qué merece conservarse.

Cinco técnicas de compresión

1. Resumen extractivo

El resumen extractivo selecciona y conserva fragmentos completos del texto original —frases o párrafos— ordenándolos para construir una versión más corta sin modificar su redacción.

Cuándo utilizarlo: cuando la fidelidad literal importa y no se puede permitir que el resumen reformule o interprete el contenido. Útil para contratos, especificaciones técnicas y declaraciones formales.

Limitaciones: puede resultar redundante si los fragmentos seleccionados se solapan, y su resultado puede ser difícil de leer si los fragmentos pertenecen a partes distantes del texto original.

2. Resumen abstractivo

El resumen abstractivo genera nuevas oraciones que capturan el significado del texto original sin copiar fragmentos. El modelo interpreta, reordena y reformula para producir una versión más concisa.

Cuándo utilizarlo: cuando se necesita reducir significativamente la longitud del contexto y se acepta cierto grado de interpretación. Adecuado para resumir conversaciones, reuniones o sesiones de trabajo.

Limitaciones: puede introducir imprecisiones o perder matices que sí estaban en el original. Requiere validar la calidad del resumen, especialmente en dominios técnicos.

3. Resumen jerárquico

El resumen jerárquico organiza el contenido en niveles: primero resume bloques pequeños, luego agrupa esos resúmenes en bloques más grandes, repitiendo el proceso hasta obtener una versión manejable.

Cuándo utilizarlo: cuando el volumen de texto es muy grande —documentos extensos, sesiones de muchos días— y un resumen plano perdería demasiado detalle. Permite mantener múltiples niveles de granularidad.

Limitaciones: más costoso en términos computacionales y de tokens de procesamiento. Requiere una estructura lógica en el texto original para que los niveles resulten coherentes.

4. Resumen incremental

El resumen incremental actualiza un resumen existente cada vez que se incorporan nuevos mensajes, en lugar de regenerarlo desde cero.

El proceso es sencillo: al finalizar cada turno de conversación, se envía el resumen anterior junto con los nuevos intercambios y se genera una versión actualizada que reemplaza a la anterior.

Cuándo utilizarlo: en conversaciones largas de sesión continua donde es necesario mantener el estado actualizado sin reprocessar toda la historia.

Limitaciones: si el resumen base tiene errores, estos se propagan y pueden amplificarse. Conviene auditar el resumen periódicamente, no solo actualizarlo.

5. Compresión semántica

La compresión semántica elimina tokens de baja información —repeticiones, frases de relleno, saludos, cortesías— preservando el contenido semántico esencial. En su forma más sofisticada, puede reescribir oraciones completas para expresar el mismo significado con menos tokens.

Cuándo utilizarlo: cuando las instrucciones del sistema o los documentos recuperados son verbosos y contienen información secundaria que no aporta al razonamiento del modelo.

Limitaciones: requiere criterio para decidir qué es "baja información". Una compresión agresiva puede eliminar contexto que resulta relevante en situaciones inesperadas.

Criterio de selección

La elección de la técnica depende de tres variables:

Variable	Pregunta orientadora
Fidelidad requerida	¿El modelo necesita las palabras exactas del original?
Volumen de texto	¿Cuánto hay que reducir y en cuántas iteraciones?
Frecuencia de actualización	¿El resumen se genera una vez o se actualiza continuamente?

En la práctica, las arquitecturas maduras combinan varias técnicas: resumen incremental para el historial conversacional, resumen jerárquico para bases documentales, y compresión semántica para el system prompt.

Buenas prácticas

- Resumir antes de alcanzar el límite, no en reacción a él.
 - Validar periódicamente la calidad del resumen enviando al modelo preguntas de verificación sobre el contenido resumido.
 - Conservar siempre hechos, decisiones adoptadas y compromisos asumidos; son los datos de mayor valor en cualquier conversación.
 - Eliminar redundancias antes de aplicar cualquier técnica de resumen: un texto limpio produce mejores resúmenes.
 - Documentar la estrategia de resumido en el código, especificando qué se conserva, qué se descarta y con qué frecuencia se actualiza.
-

Resumen

Elegir la técnica de compresión correcta es una decisión de diseño que afecta la calidad de las respuestas, el costo operativo y la continuidad de la experiencia del usuario. No existe una técnica universalmente superior: la elección depende del tipo de contenido, la frecuencia de actualización y el nivel de fidelidad requerido.

En la próxima sección estudiaremos estrategias concretas para optimizar el consumo total de tokens a lo largo de toda la arquitectura de la aplicación.

Estrategias para optimizar el consumo de tokens

Módulo 3 — Context Engineering Profesional

Introducción

Optimizar tokens no es solo una medida de ahorro económico. Reducir el consumo mejora la latencia de respuesta, libera espacio para información de mayor valor y hace que el razonamiento del modelo sea más preciso al eliminar el ruido del contexto.

Las estrategias de optimización operan en dos niveles: el nivel del prompt —qué se incluye y cómo se formula— y el nivel de la arquitectura —cómo se selecciona, almacena y recupera la información antes de enviarla al modelo.

Estrategias en el nivel del prompt

Eliminar contexto redundante

Antes de enviar una solicitud, conviene revisar si el contexto contiene información repetida. El mismo dato puede aparecer en las instrucciones del sistema, en un documento recuperado y en el historial conversacional simultáneamente. El modelo no gana claridad por recibir tres veces el mismo hecho; solo pierde espacio disponible.

Una política útil: si un dato ya está en el system prompt, no lo repita en los documentos RAG. Si está en los documentos, no lo reformule en las instrucciones.

Recuperar solo los documentos relevantes

Los sistemas RAG pueden devolver cinco, diez o veinte documentos por consulta. Enviarlos todos al modelo por precaución es un error habitual. Cada documento que no aporta al razonamiento consume tokens y puede diluir la atención del modelo sobre los que sí importan.

La solución es diseñar el proceso de recuperación con umbral de relevancia: solo incluir documentos cuya similitud supere un valor mínimo. En la práctica, dos o tres documentos de alta relevancia superan en efectividad a diez de relevancia media.

Limitar los ejemplos few-shot

Los ejemplos few-shot tienen un costo fijo en tokens que se paga en cada solicitud. Si el modelo ya tiene suficiente capacidad para la tarea sin ejemplos —o si los ejemplos ya están incorporados en el fine-tuning— incluirlos es un gasto innecesario.

La práctica recomendada es comenzar sin ejemplos, medir la calidad de las respuestas y agregar ejemplos solo si el rendimiento no es aceptable. Menos ejemplos bien elegidos son más efectivos que muchos ejemplos genéricos.

Normalizar respuestas de herramientas

Los resultados devueltos por herramientas externas —APIs, bases de datos, ejecutores de código— pueden ser muy verbosos: respuestas JSON completas, encabezados HTTP, mensajes de debug. Enviarlos tal cual al modelo desperdicia tokens en información que no aporta al razonamiento.

La solución es un paso de normalización: extraer solo los campos relevantes, convertir JSON complejo en texto estructurado conciso y eliminar metadatos que el modelo no necesita.

Mantener instrucciones compactas

El system prompt tiende a crecer con el tiempo a medida que se agregan nuevas reglas, restricciones y comportamientos esperados. Una auditoría periódica del system prompt suele revelar instrucciones redundantes, contradictorias o ya superadas por otras.

Un system prompt eficiente es aquel que contiene solo las instrucciones que el modelo no puede inferir de su entrenamiento o del contexto de la conversación.

Estrategias en el nivel de la arquitectura

Caching de contexto

Los principales proveedores ofrecen mecanismos de **caching de prefijos** que permiten reutilizar partes del contexto entre solicitudes sin volver a procesarlas. Cuando una porción del contexto —como el system prompt o una base de conocimiento estática— permanece igual entre consultas, el proveedor puede computarla una vez y cobrar una fracción del precio en las solicitudes subsiguientes.

Esta técnica es especialmente rentable en aplicaciones con system prompts extensos o con documentos de referencia que se incorporan a cada consulta. El ahorro puede ser sustancial cuando el volumen de solicitudes es alto.

La implementación práctica varía según el proveedor: algunos lo hacen automáticamente cuando detectan prefijos repetidos, otros requieren que el sistema marque explícitamente qué fragmentos son candidatos al cache.

Separar historial, memoria y RAG

Mezclar tres fuentes de información distintas en un único bloque de texto aumenta el consumo de tokens y complica el razonamiento del modelo. Una arquitectura bien diseñada mantiene separados:

- el **historial conversacional**: los últimos N intercambios del turno actual;
- la **memoria persistente**: hechos y preferencias de largo plazo del usuario;
- los **documentos RAG**: información recuperada específicamente para esta consulta.

Esta separación permite aplicar políticas de optimización distintas a cada capa, y al modelo le resulta más sencillo discriminar entre qué es contexto inmediato y qué es conocimiento de base.

Herramientas para medir tokens

Optimizar sin medir es opinar sin evidencia. Las herramientas de conteo de tokens permiten instrumentar la aplicación y detectar dónde se concentra el consumo.

Anthropic: la API expone el campo `usage` en cada respuesta, que devuelve `input_tokens` y `output_tokens`. La biblioteca oficial también ofrece el método `count_tokens` para estimar el consumo antes de enviar la solicitud.

OpenAI: la biblioteca `tiktoken` permite contar tokens localmente para todos los modelos de la familia GPT, sin necesidad de hacer una llamada a la API.

Herramientas de observabilidad: plataformas como LangSmith, Helicone o Phoenix permiten registrar y visualizar el consumo de tokens por sesión, por usuario y por tipo de consulta, facilitando la detección de patrones de uso ineficiente.

La instrumentación mínima recomendada para cualquier aplicación en producción es registrar, para cada solicitud: tokens de entrada, tokens de salida, costo estimado y latencia. Con esos cuatro datos es posible detectar regresiones y oportunidades de optimización de forma continua.

Métricas para el seguimiento operativo

El seguimiento sistemático del consumo de tokens requiere definir métricas concretas:

- **Tokens promedio por solicitud:** establece la línea base de consumo y permite detectar derivaciones hacia arriba que indican acumulación de contexto no gestionada.
 - **Ratio input/output:** una relación muy alta de tokens de salida respecto a los de entrada puede indicar que el modelo está generando respuestas más extensas de lo necesario.
 - **Costo por conversación:** calcula el costo total de una sesión completa, no solo de cada solicitud individual. Permite comparar estrategias de administración del contexto.
 - **Tiempo de respuesta:** correlaciona el consumo de tokens con la latencia percibida por el usuario.
-

Buenas prácticas

- Instrumentar el consumo de tokens desde el primer día de desarrollo, no al detectar problemas en producción.
 - Revisar el system prompt cada tres meses para eliminar instrucciones obsoletas.
 - Establecer un umbral de tokens de entrada máximo por solicitud y alertar cuando se supere.
 - Aplicar normalización a todas las respuestas de herramientas externas antes de incluirlas en el contexto.
 - Evaluar la posibilidad de usar caching de prefijos en aplicaciones con system prompts extensos o bases de conocimiento estáticas.
-

Resumen

La optimización de tokens opera en múltiples niveles simultáneamente: el contenido del prompt, la arquitectura de recuperación y las capacidades de la plataforma. Las aplicaciones que miden, controlan y optimizan el consumo de tokens de forma sistemática no solo reducen costos sino que, en la mayoría de los casos, también mejoran la calidad de las respuestas al eliminar el ruido del contexto.

En la próxima sección estudiaremos los patrones arquitectónicos más importantes para administrar el contexto en aplicaciones de producción.

Patrones de administración del contexto

Módulo 3 — Context Engineering Profesional

Introducción

Ninguna estrategia de administración del contexto sirve para todos los casos. La elección del patrón correcto depende del tipo de aplicación, la naturaleza del conocimiento que debe gestionarse, el presupuesto disponible y la complejidad que el equipo puede sostener en producción.

En esta sección describimos los cuatro patrones arquitectónicos más utilizados en aplicaciones profesionales. Cada uno define una forma distinta de organizar el contexto: qué se conserva, qué se descarta, qué se recupera y cómo se combinan las distintas fuentes de información.

Patrón 1: Sliding Window (Ventana deslizante)

Descripción

La Sliding Window conserva únicamente las últimas N interacciones de la conversación. Cuando se agrega un nuevo mensaje, el más antiguo del conjunto se elimina para mantener el tamaño del contexto constante.

```
Solicitud N: [sistema] [msg-6] [msg-7] [msg-8] [usuario]
Solicitud N+1: [sistema] [msg-7] [msg-8] [nuevo] [usuario]
```

El system prompt permanece fijo; solo el historial rota.

Cuándo utilizarlo

- Conversaciones de soporte o atención donde cada consulta es mayoritariamente independiente.
- Aplicaciones de chat general donde el contexto reciente es más relevante que el histórico.
- Prototipos y sistemas que necesitan una solución simple y predecible.

Ventajas

- Implementación trivial: solo requiere mantener una lista y eliminar el elemento más antiguo.
- Costo predecible: el tamaño del contexto nunca crece más allá de N mensajes.
- Latencia estable: no depende de operaciones externas de recuperación.

Compromisos

- Pierde información histórica. Si el usuario retoma un tema tratado hace veinte turnos, el modelo no tendrá acceso a ese contexto.

- No escala bien para sesiones de larga duración donde la continuidad histórica es importante.
 - La elección del valor N implica un equilibrio entre cobertura histórica y costo.
-

Patrón 2: Summary + Window (Resumen más ventana reciente)

Descripción

Este patrón combina un bloque de resumen histórico con una ventana de los mensajes más recientes. El resumen se actualiza periódicamente —al final de cada sesión, cada N mensajes o cuando el historial supera un umbral— y captura las decisiones, hechos y compromisos acumulados.

```
Contexto: [sistema] [resumen-histórico] [msg-reciente-1] [msg-reciente-2] [usuario]
```

El resumen actúa como memoria de trabajo comprimida; los mensajes recientes aportan el contexto inmediato.

Cuándo utilizarlo

- Asistentes empresariales con sesiones de trabajo continuas a lo largo de días o semanas.
- Agentes que gestionan proyectos o tareas de larga duración.
- Cualquier aplicación donde la continuidad histórica importa pero el historial completo no cabe en la ventana.

Ventajas

- Equilibra continuidad histórica con eficiencia de tokens.
- Permite conversaciones de duración indefinida sin perder el hilo.
- El resumen puede auditarse y corregirse manualmente en aplicaciones críticas.

Compromisos

- La calidad del sistema depende de la calidad del resumen. Un resumen deficiente propaga errores.
 - Requiere decidir cuándo y cómo actualizar el resumen, lo que agrega complejidad al ciclo de vida de la sesión.
 - El resumen mismo consume tokens; si es demasiado detallado, pierde su ventaja sobre el historial completo.
-

Patrón 3: RAG First (Recuperación prioritaria)

Descripción

En el patrón RAG First, el conocimiento permanente de la aplicación —manuales, políticas, documentación técnica, bases de datos— no reside en el contexto de forma fija. Se recupera en tiempo real para cada consulta, seleccionando solo los fragmentos relevantes según el contenido del mensaje del usuario.

Consulta del usuario

|

▼

Sistema de recuperación (embeddings + búsqueda)

|

▼

Top-K documentos relevantes

|

▼

Contexto: [sistema] [doc-1] [doc-2] [historial-reciente] [usuario]

El historial conversacional puede combinarse con los documentos recuperados o administrarse por separado.

Cuándo utilizarlo

- Aplicaciones de búsqueda sobre bases de conocimiento extensas: manuales técnicos, repositorios de políticas, catálogos de productos.
- Sistemas donde la documentación cambia con frecuencia y mantenerla en el system prompt sería impracticable.
- Cualquier caso donde el conocimiento relevante para una consulta es una fracción pequeña de un conjunto grande.

Ventajas

- El contexto siempre contiene información actualizada: modificar la base de conocimiento no requiere cambiar el system prompt.
- Eficiente en tokens: solo se incluye lo relevante para cada consulta específica.
- Escalable a bases de conocimiento de cualquier tamaño.

Compromisos

- La calidad de las respuestas depende de la calidad de la recuperación. Si los documentos recuperados no son los correctos, el modelo no podrá responder bien aunque la respuesta exista en la base.
 - Agrega latencia por la operación de búsqueda y la distancia de embeddings.
 - Requiere mantener un índice vectorial actualizado y un pipeline de indexación.
-

Patrón 4: Memoria + Historial + RAG (Arquitectura de tres capas)

Descripción

Este patrón combina los tres mecanismos anteriores asignando una responsabilidad específica y diferenciada a cada capa del contexto:

- **Memoria persistente:** hechos duraderos sobre el usuario, sus preferencias, su historial de interacciones anteriores. Se recupera al inicio de cada sesión.
- **Historial reciente:** los últimos mensajes del turno actual, con o sin resumen histórico.
- **Documentos RAG:** conocimiento técnico o documental recuperado en tiempo real para la consulta actual.

```
Contexto: [sistema] [memoria-usuario] [resumen] [docs-RAG] [historial-reciente] [usuari
```

Cada capa tiene su propio almacenamiento, su propia política de actualización y su propio ciclo de vida.

Cuándo utilizarlo

- Asistentes de alto valor donde la personalización y la continuidad a largo plazo son requisitos del negocio.
- Agentes empresariales que coordinan múltiples dominios de conocimiento.
- Aplicaciones donde el usuario espera que el sistema "recuerde" quién es y qué ha pedido antes, al mismo tiempo que accede a documentación actualizada.

Ventajas

- Ofrece la mayor continuidad y personalización posible.
- Cada capa puede optimizarse independientemente.

- Separa claramente las responsabilidades, lo que facilita el mantenimiento y la auditoría.

Compromisos

- Es el patrón más complejo de implementar y operar.
 - Requiere tres sistemas de almacenamiento distintos: base de memoria, sistema de vectores para RAG y almacén de historial.
 - La coordinación entre capas puede introducir inconsistencias si no se gestiona correctamente.
 - El costo operativo es significativamente mayor que los patrones simples.
-

Guía de selección

Caso de uso	Patrón recomendado
Chat simple o prototipo	Sliding Window
Conversaciones de larga duración	Summary + Window
Búsqueda sobre base de conocimiento	RAG First
Asistente empresarial personalizado	Memoria + Historial + RAG

En la práctica, muchos sistemas evolucionan desde el patrón más simple hacia el más complejo a medida que los requisitos del negocio lo demandan. Comenzar con Sliding Window y agregar capas progresivamente es una estrategia de desarrollo sostenible.

Buenas prácticas

- Elegir el patrón más simple que satisfaga los requisitos: la complejidad adicional tiene costos de desarrollo y operación reales.
 - Documentar explícitamente qué capa del contexto es responsable de cada tipo de información.
 - Implementar monitoreo independiente para cada capa: detectar qué capa falla cuando la calidad degrada.
 - Planificar la evolución del patrón desde el inicio: un sistema diseñado solo para Sliding Window puede ser difícil de migrar a un patrón de tres capas sin refactorización importante.
-

Resumen

Los cuatro patrones de administración del contexto —Sliding Window, Summary + Window, RAG First y Memoria + Historial + RAG— no son alternativas excluyentes en un catálogo fijo. Son puntos de referencia en un espacio de diseño continuo. La elección correcta depende del dominio, el presupuesto, la complejidad de los requisitos y la madurez del equipo.

En la próxima sección aplicaremos estos conceptos en un ejercicio práctico de diseño de contexto para una aplicación empresarial real.

Laboratorio práctico: diseño de contexto para aplicaciones empresariales

Módulo 3 — Context Engineering Profesional

Introducción

Los conceptos de este capítulo —tokens, tokenización, ventanas de contexto, técnicas de compresión y patrones de administración— cobran significado real cuando se aplican a una situación concreta.

Este laboratorio propone tres ejercicios progresivos. Cada uno tiene un objetivo técnico específico y puede resolverse con papel y lápiz, con código o con llamadas directas a la API del modelo.

Ejercicio 1: Estimación del consumo de contexto

Escenario

Un equipo está desarrollando un asistente de soporte interno para una empresa de software. El system prompt actual tiene aproximadamente 800 palabras. El asistente mantiene en contexto las últimas 10 interacciones del usuario (promedio de 60 palabras por mensaje, entre usuario y modelo). En cada consulta, el sistema RAG recupera tres fragmentos de documentación técnica de 300 palabras cada uno.

Tarea

1. Estime la cantidad de tokens de entrada por solicitud, considerando que el español tiene aproximadamente 1,3 tokens por palabra en la mayoría de los tokenizadores modernos.
2. Identifique qué capa del contexto consume más tokens.
3. Proponga una estrategia para reducir el consumo total en al menos un 30 % sin degradar la calidad de las respuestas.

Referencia de cálculo

Capa	Palabras	Tokens estimados
System prompt	800	~1.040
Historial (10 intercambios)	600	~780
Documentos RAG (3 x 300)	900	~1.170
Total estimado	2.300	~2.990

Una reducción del 30 % en los documentos RAG —pasando de tres fragmentos de 300 palabras a dos fragmentos de 200 palabras cada uno— reduce el consumo de esa capa en más

de un 55 %, llevando el total de entrada a aproximadamente 2.050 tokens.

Ejercicio 2: Selección del patrón de administración

Escenarios

Para cada uno de los siguientes casos, seleccione el patrón de administración más adecuado entre los cuatro estudiados en la sección anterior (Sliding Window, Summary + Window, RAG First, Memoria + Historial + RAG) y justifique la elección en dos o tres oraciones.

Caso A: Una empresa de e-commerce quiere implementar un chatbot de preguntas frecuentes sobre sus productos. El catálogo tiene 5.000 productos con especificaciones técnicas. Las consultas son mayoritariamente independientes entre sí.

Caso B: Un despacho legal usa un asistente para gestionar un caso judicial que se extiende durante ocho meses. El asistente debe recordar todas las decisiones procesales, los documentos analizados y las estrategias acordadas.

Caso C: Una plataforma de e-learning ofrece un tutor de programación. El tutor debe conocer el nivel actual del estudiante y su historial de errores anteriores, y al mismo tiempo consultar la documentación técnica del lenguaje que se está enseñando.

Respuestas de referencia

Caso A: RAG First. Las consultas son independientes, el catálogo es extenso y cambia con frecuencia. No hay necesidad de continuidad histórica entre sesiones. La recuperación en tiempo real garantiza que las especificaciones estén actualizadas.

Caso B: Summary + Window. El caso dura meses y la continuidad histórica es fundamental. Un resumen estructurado de las decisiones procesales reemplaza el historial completo sin perder los hechos relevantes. La ventana reciente permite trabajar sobre los documentos del día.

Caso C: Memoria + Historial + RAG. El tutor necesita tres capas: la memoria persistente del estudiante (nivel, errores frecuentes, temas cubiertos), el historial reciente de la sesión actual

y los documentos técnicos del lenguaje recuperados por RAG. Ninguna de las otras opciones proporciona las tres capacidades simultáneamente.

Ejercicio 3: Auditoría de un system prompt

Tarea

Analice el siguiente system prompt y aplique compresión semántica para reducir su longitud sin perder información operativa relevante.

System prompt original (versión a auditar):

Eres un asistente de soporte técnico para la empresa Acme. Tu función principal y más importante es ayudar a los usuarios que trabajan en la empresa con sus problemas técnicos de computadoras, software y equipos de oficina. Debes ser siempre muy amable, cortés y respetuoso con todos los usuarios en todo momento. Cuando un usuario te haga una pregunta, debes escucharla con atención y tratar de entenderla bien antes de responder. Si no sabes la respuesta a una pregunta, debes decírselo honestamente al usuario y sugerirle que contacte al equipo de soporte de nivel dos. Nunca debes inventar información ni dar datos incorrectos. Responde siempre en español. Mantén tus respuestas breves y al punto, sin información innecesaria. No discutas temas que no estén relacionados con soporte técnico.

Versión comprimida (referencia):

Eres el asistente de soporte técnico de Acme. Ayudas a empleados con problemas de hardware, software y equipos de oficina. Sé amable y directo. Si no conoces la respuesta, déralo al soporte de nivel dos. No inventes información. Responde siempre en español y fuera del ámbito de soporte técnico.

La versión original tiene aproximadamente 135 palabras (~175 tokens). La versión comprimida tiene aproximadamente 55 palabras (~72 tokens), una reducción del 59 % sin pérdida de instrucciones operativas.

Lista de verificación del arquitecto

Antes de desplegar una aplicación basada en LLM, verifique:

- ☐ El contexto contiene únicamente información relevante para la consulta actual.
 - ☐ Existe una política explícita de descarte y resumido documentada en el código.
 - ☐ El consumo de tokens se registra en cada solicitud: entrada, salida y costo estimado.
 - ☐ Las capas de historial, memoria y RAG están claramente separadas y no duplican información.
 - ☐ El system prompt ha sido auditado en los últimos tres meses.
 - ☐ El patrón de administración elegido está documentado y el equipo conoce sus compromisos.
 - ☐ Se ha evaluado la posibilidad de aplicar caching de prefijos para reducir costos en solicitudes frecuentes.
-

Errores frecuentes

- **Llenar la ventana por precaución:** incorporar documentos o historial "por si acaso" degrada el razonamiento y aumenta el costo sin beneficio proporcional.
 - **Duplicar información entre capas:** el mismo dato en el system prompt, en la memoria y en el RAG no aporta valor al modelo; solo consume tokens.
 - **No medir costos:** operar sin instrumentación hace imposible detectar regresiones ni evaluar el impacto de cambios en la arquitectura del contexto.
 - **Confiar exclusivamente en el historial:** una aplicación que depende solo del historial conversacional se vuelve inoperable a medida que la conversación crece.
 - **Actualizar el resumen demasiado tarde:** resumir después de haber superado el límite obliga a reconstruir el contexto desde cero, con el riesgo de perder información relevante.
-

Resumen

La ingeniería del contexto es, en último término, una disciplina de diseño. Los conceptos de este capítulo —tokens, ventanas, compresión, patrones— son herramientas. La habilidad del AI Engineer está en combinarlas correctamente para cada situación específica.

En la próxima sección cerraremos el capítulo con una síntesis de los conceptos clave y la transición hacia el diseño de sistemas de memoria, que es el paso natural después de haber comprendido los límites y la gestión de la ventana de contexto.

Cierre del capítulo: ventanas de contexto y gestión de tokens

Módulo 3 — Context Engineering Profesional

Síntesis del capítulo

Este capítulo recorrió los conceptos fundamentales que determinan la capacidad y el rendimiento de cualquier aplicación basada en LLM.

Empezamos por entender que la **ventana de contexto** es el espacio de trabajo del modelo: todo lo que entra en ella —instrucciones, historial, memoria, documentos, resultados de herramientas— compite por un recurso limitado. Administrar ese espacio es una responsabilidad explícita del AI Engineer, no una tarea delegable al modelo.

Aprendimos que la unidad de medida de ese espacio es el **token**, una representación que no equivale a palabras ni a caracteres. Los tokens de entrada y los de salida tienen costos distintos; ambos deben medirse. La **tokenización** varía según el modelo y el tokenizador, por lo que las estimaciones deben hacerse sobre el sistema concreto que se va a utilizar.

Estudiamos cómo **evolucionaron las ventanas de contexto** y por qué un mayor tamaño no garantiza mejores resultados: el fenómeno "lost in the middle" demuestra que la posición de la información dentro del contexto importa tanto como su presencia. La calidad del contexto siempre precede a su cantidad.

Analizamos qué ocurre cuando el **contexto supera el límite** disponible y las cuatro estrategias para administrarlo: descarte selectivo, resumido, recuperación bajo demanda y memoria persistente.

Desarrollamos las **técnicas de compresión** —extractiva, abstractiva, jerárquica, incremental y semántica— con sus casos de uso, ventajas y limitaciones. Cada técnica responde a un tipo diferente de problema; la elección depende del tipo de contenido, la frecuencia de actualización y el nivel de fidelidad requerido.

Exploramos las **estrategias de optimización de tokens** tanto en el nivel del prompt como en el nivel de la arquitectura, incluyendo el caching de prefijos y las herramientas de medición disponibles en los principales proveedores.

Describimos los **cuatro patrones arquitectónicos** de administración del contexto —Sliding Window, Summary + Window, RAG First y Memoria + Historial + RAG— con sus criterios de selección, ventajas y compromisos.

Finalmente, aplicamos todos los conceptos en un **laboratorio práctico** con ejercicios de estimación, selección de patrones y auditoría de prompts.

Autoevaluación

Las siguientes preguntas permiten verificar la comprensión de los conceptos del capítulo.

1. ¿Qué diferencia existe entre palabras y tokens?

Un token es la unidad mínima de procesamiento del modelo, y no equivale a una palabra ni a un carácter. Una palabra puede generar uno o varios tokens dependiendo del tokenizador; una misma frase produce cantidades distintas de tokens en modelos diferentes. No existe una tasa de conversión fija.

2. ¿Por qué una ventana más grande no garantiza mejores respuestas?

Por dos razones principales. La primera es el fenómeno "lost in the middle": los modelos prestan menos atención a la información ubicada en el centro del contexto, de modo que agregar más texto no asegura que el modelo lo procese con igual eficacia. La segunda es que más información sin filtrar introduce ruido, y el razonamiento del modelo sobre un contexto de calidad reducida tiende a ser menos preciso.

3. ¿Cuándo conviene resumir el contexto?

Antes de alcanzar el límite del modelo, no después. La política de resumido debe activarse a medida que el historial crece, de forma incremental y controlada. Resumir en reacción a un error de límite excedido obliga a reconstruir el contexto con riesgo de pérdida de información.

4. ¿Qué patrón utilizaría para un asistente empresarial que trabaja con usuarios durante semanas y necesita consultar documentación técnica actualizada?

El patrón Memoria + Historial + RAG es el más adecuado. La memoria persistente conserva el perfil y el historial a largo plazo del usuario; el historial reciente más un resumen captura el contexto de la sesión actual; y el RAG recupera en tiempo real los documentos técnicos relevantes para cada consulta. Ninguno de los tres mecanismos por separado cumple los tres requisitos simultáneamente.

Checklist del arquitecto

Antes de pasar al siguiente capítulo, verifique que puede responder afirmativamente a cada uno de los siguientes puntos:

- ☐ Comprendo qué es un token y por qué no equivale a una palabra.
 - ☐ Sé distinguir los tokens de entrada de los de salida y entiendo su impacto en el costo.
 - ☐ Puedo estimar el consumo de tokens de una solicitud típica de mi aplicación.
 - ☐ Conozco el fenómeno "lost in the middle" y sé cómo mitigarlo en el diseño del contexto.
 - ☐ Tengo clara la diferencia entre descarte, resumido, recuperación bajo demanda y memoria persistente.
 - ☐ Puedo elegir entre las cinco técnicas de compresión según el tipo de contenido y los requisitos.
 - ☐ Conozco los cuatro patrones de administración y sé cuándo aplicar cada uno.
 - ☐ Mi aplicación tiene o tendrá instrumentación para registrar el consumo de tokens en producción.
-

Hacia el siguiente capítulo

El capítulo 03 nos dio las herramientas para administrar lo que el modelo puede ver en un momento dado. Pero una pregunta queda abierta: ¿qué ocurre con la información que necesita persistir más allá de la ventana de contexto, entre sesiones, a lo largo del tiempo?

La respuesta está en el diseño de la **memoria**. Una vez que comprendemos los límites de la ventana —cuánto cabe, cuánto cuesta, qué se pierde y por qué— el paso natural es profundizar en uno de sus componentes más complejos: cómo construir sistemas que recuerdan, que aprenden del historial del usuario y que mantienen estado entre sesiones separadas por horas, días o semanas.

El capítulo 04 abordará la memoria de corto y largo plazo y las estrategias avanzadas para construir asistentes persistentes.